

Lab 5 - API, Auth, Tests

Predaja: 12.6.

Zadaci i bodovanje

Kriterij	Bodovi
Implementirati kompletnu API podršku za sve entitete (CRUD, DTO)	2
Omogućiti autentikaciju (local accounts) i autorizaciju	1
Omogućiti upload datoteka (dropzone ili sl.)	1
Omogućiti 3rd party autentikaciju (Google, FB, ...)	1
Implementirati integracijske testove za API endpointe (sve, CRUD)	2

Nužni uvjeti za predaju vježbe

☐ Implementirati kompletnu API podršku za sve entitete (CRUD, DTO) (2 boda)

- ☐ API controlleri moraju podržavati osnovne CRUD operacije za sve entitete gdje poslovna pravila to dopuštaju
 - ☐ **GET** za dohvat svih zapisa, uz opciju pretrage (query i jos ponesto)
 - ☐ **GET** za dohvat jednog zapisa po ID-u
 - ☐ **POST** za kreiranje zapisa
 - ☐ **PUT** za izmjenu zapisa
 - ☐ **DELETE** za brisanje zapisa
- ☐ API ne smije direktno izlagati nepotrebna interna polja entiteta
 - ☐ Koristiti DTO klase za podatke koji se vraćaju API klijentu
 - ☐ Povezane podatke prikazati kroz ugniježdene DTO klase gdje ima smisla
- ☐ Upload datoteka mora biti vezan uz konkretnog kviza
 - ☐ Upload raditi asinkrono preko Dropzone komponente ili održavane alternative
 - ☐ Datoteke spremiti na disk

- ☐ U bazu spremiti metapodatke i putanju
- ☐ Popis datoteka učitati AJAX pozivom
- ☐ Omogućiti brisanje postojećih datoteka
- ☐ Autentikacija mora biti uključena kroz ASP.NET Core Identity
 - ☐ Lokalna registracija i prijava moraju raditi
 - ☐ `AppUser` mora biti proširen traženim poljima
 - ☐ Autorizacija mora ograničavati akcije prema pravilima zadatka
 - ☐ Javne akcije dostupne su anonimnim korisnicima
 - ☐ Create/Edit/Delete dostupni su samo dopuštenim korisnicima
 - ☐ Role `Admin` i barem još jedna rola moraju biti implementirane
- ☐ Google ili Facebook login mora raditi
- ☐ Integracijski testovi moraju pokriti API endpointe za sve CRUD operacije
 - ☐ Testirati uspješne scenarije
 - ☐ Testirati nepostojeće ID-eve
 - ☐ Testirati validacijske pogreške gdje postoje

Web API

ASP.NET Web API je mehanizam kojim aplikacija izlaže podatke i operacije drugim klijentima. Za razliku od klasičnog MVC pristupa, gdje controller najčešće vraća cijelu HTML stranicu, API najčešće vraća strukturirane podatke, primjerice JSON.

U .NET Core i novijim verzijama nema zasebne `ApiController` klase kao u starijem ASP.NET MVC-u. Controller je i dalje obična controller klasa, ali se za API scenarije u pravilu označava atributom `[ApiController]`.

Tipični scenariji korištenja API-ja:

- JavaScript u web aplikaciji poziva API i prikazuje podatke korisniku
- Jedan server komunicira s drugim serverom
- Mobilna ili desktop aplikacija koristi backend aplikacije
- Integracijski testovi pozivaju API endpointe i provjeravaju rezultate

API pozivi najčešće prate HTTP metode:

- `GET` — dohvat podataka
- `POST` — kreiranje novog zapisa
- `PUT` — izmjena postojećeg zapisa
- `DELETE` — brisanje zapisa

U praksi se mogu vidjeti i drugi HTTP methodi, primjerice `PATCH`, ali za ovu vježbu dovoljni su `GET`, `POST`, `PUT` i `DELETE`. U starijim ili legacy sustavima nije neobično vidjeti samo `GET` i `POST`, pri čemu se `POST` koristi i za kreiranje, izmjenu i brisanje. Za ovu vježbu ipak je bolje koristiti jasne metode jer iz samog zahtjeva odmah vidimo namjeru.

HTTP status kodovi

API ne vraća samo podatke, nego i HTTP status kod. Status kod klijentu govori je li zahtjev uspio, je li korisnik neautoriziran, je li zapis pronađen ili se dogodila greška.

Najčešći status kodovi:

- `200 OK` — zahtjev je uspješno obrađen
- `201 Created` — novi resurs je uspješno kreiran
- `204 No Content` — zahtjev je uspio, ali odgovor nema tijelo
- `400 Bad Request` — zahtjev nije ispravan, često zbog validacije
- `401 Unauthorized` — korisnik nije autentificiran ili token nije valjan
- `403 Forbidden` — korisnik je poznat, ali nema pravo pristupa
- `404 Not Found` — traženi zapis ne postoji
- `422 Unprocessable Entity` — zahtjev je sintaktički ispravan, ali ga nije moguće obraditi
- `429 Too Many Requests` — klijent je poslao previše zahtjeva u kratkom vremenu
- `503 Service Unavailable` — servis trenutno nije dostupan

Status kodovi se često grupiraju:

- `2xx` — uspjeh
- `3xx` — redirect
- `4xx` — očekivane greške na strani klijenta, koje klijent može popraviti

- `5xx` — neočekivane greške na strani servera

Autentikacija kod MVC stranica i API-ja

Klasične MVC stranice najčešće koriste authentication cookie. Nakon prijave browser šalje cookie pri svakom sljedećem zahtjevu, a aplikacija preko njega zna tko je korisnik.

API-ji se često koriste iz JavaScript klijenata, mobilnih aplikacija, desktop aplikacija ili server-to-server integracija. Zato se za API-je često koriste bearer tokeni, primjerice JWT ili access token. Cookie autentikacija tehnički može raditi i za API, ali nije najčešći obrazac za javne ili integracijske API-je.



U .NET Core MVC aplikaciji isti projekt može istovremeno imati klasične MVC stranice i API controllere. Razlika je primarno u tome što MVC akcije vraćaju viewove, a API akcije vraćaju podatke.

API controller

Za API controller potrebno je definirati rutu kojom će se do njega dolaziti.

Primjer za kvizove:

`QuizApiController.cs`

```
[Route("api/quiz")]
[ApiController]
public class QuizApiController : ControllerBase
{
    private readonly QuizManagerDbContext _dbContext;

    public QuizApiController(QuizManagerDbContext dbContext)
    {
        this._dbContext = dbContext;
    }
}
```

U starijim primjerima može se vidjeti nasljeđivanje iz `Controller`, ali za čisti API controller praktičnije je koristiti `ControllerBase`, jer API controlleru ne trebaju MVC view funkcionalnosti.

`[ApiController]` nije samo oznaka za čitljivost koda. U ASP.NET Core aplikaciji donosi nekoliko korisnih ponašanja za API scenarije:

- očekuje attribute routing, primjerice `[Route("api/quiz")]`
- poboljšava model binding za API zahtjeve
- automatski vraća `400 Bad Request` kada je `ModelState` neispravan
- proizvodi API-friendly odgovore za validacijske greške

Zbog toga je za API controllere dobro koristiti `[ApiController]`, čak i kada običan MVC controller tehnički može vratiti JSON.

DTO klase

DTO (`Data Transfer Object`) je klasa koja definira oblik podataka koji API vraća ili prima. Iako je tehnički moguće direktno vraćati Entity Framework entitete, to često nije dobro rješenje.

Razlozi za korištenje DTO klasa:

- entitet može sadržavati interna polja koja ne smiju ići na frontend
- entitet može imati navigacijska svojstva koja izazovu prevelik ili ciklički JSON
- API model može biti stabilniji od baze podataka
- moguće je precizno kontrolirati što API klijent vidi
- moguće je prilagoditi podatke formatu koji API klijentu stvarno treba

Primjer DTO klasa:

`QuizDTO.cs`

```
public class QuizDTO
{
    public int ID { get; set; }
    public string Title { get; set; }
    public string Description { get; set; }
    public CategoryDTO Category { get; set; }
}
```

`CategoryDTO.cs`

```
public class CategoryDTO
{
    public int ID { get; set; }
    public string Name { get; set; }
}
```

Mapiranje entiteta u DTO

Najjednostavnije rješenje je mapiranje napraviti direktno u API metodi. To je dovoljno za prvi primjer, ali nije idealno ako se isti kod ponavlja.

Primjer mapiranja unutar metode:

```
private QuizDTO ToDTO(Quiz quiz)
{
    return new QuizDTO
    {
        ID = quiz.ID,
        Title = quiz.Title,
        Description = quiz.Description,
        Category = quiz.Category == null ? null : new CategoryDTO
        {
            ID = quiz.Category.ID,
            Name = quiz.Category.Name
        }
    };
}
```

Bolji pristupi za veće aplikacije:

- privatna metoda u controlleru
- extension metoda, primjerice `quiz.ToDTO()`
- posebna mapper klasa
- biblioteka poput AutoMappera



Za ovu vježbu dovoljno je napraviti jednostavno ručno mapiranje. Važno je da se mapiranje ne kopira nekontrolirano u svaku metodu.

Dohvat podataka

API metode za dohvat podataka najčešće koriste `GET`. Rezultat metode može biti jedan objekt, kolekcija objekata ili odgovarajući HTTP status.

Dohvat svih kvizova

Metoda bez parametara vraća sve kvizove. Poziva se kao:

```
GET /api/quiz
```

Primjer:

`QuizApiController.cs`

```
[HttpGet]
public ActionResult<IEnumerable<QuizDTO>> Get()
{
    var quizzes = this._dbContext.Quizzes
        .Include(c => c.Category)
        .Select(c => ToDTO(c))
        .ToList();

    return Ok(quizzes);
}
```

Ako se mapiranje ne može prevesti u SQL zbog korištenja privatne metode, prvo dohvatiti podatke, a zatim ih mapirati u memoriji:

```
[HttpGet]
public ActionResult<IEnumerable<QuizDTO>> Get()
{
    var quizzes = this._dbContext.Quizzes
        .Include(c => c.Category)
        .ToList()
        .Select(c => ToDTO(c))
}
```

```
        .ToList();

        return Ok(quizzes);
    }
}
```

Dohvat jednog kviza

Metoda s ID parametrom vraća jedan zapis. Poziva se kao:

```
GET /api/quiz/1
```

Primjer:

```
[HttpGet("{id}")]
public ActionResult<QuizDTO> Get(int id)
{
    var quiz = this._dbContext.Quizzes
        .Include(c => c.Category)
        .FirstOrDefault(c => c.ID == id);

    if (quiz == null)
    {
        return NotFound();
    }

    return Ok(ToDTO(quiz));
}
```

Pretraga kvizova

Pretraga se može definirati posebnom rutom. Primjer poziva:

```
GET /api/quiz/pretraga/povijest
```

Primjer:

```
[HttpGet("pretraga/{q}")]
public ActionResult<IEnumerable<QuizDTO>> Search(string q)
```



```

{
    var quizzes = this._dbContext.Quizzes
        .Include(c => c.Category)
        .Where(c =>
            c.Title.Contains(q) ||
            c.Description.Contains(q))
        .ToList()
        .Select(c => ToDTO(c))
        .ToList();

    return Ok(quizzes);
}

```



Kod stvarnih aplikacija treba paziti na performanse pretrage. Za veće tablice potrebno je razmisliti o indeksima, ograničenju broja rezultata i robusnijem modelu pretrage.



Podaci poslani kroz GET parametre završavaju u URL-u. Zato se lozinke, tokeni, osobni podaci i osjetljivi kriteriji pretrage ne smiju slati kroz query string ili route parametre. Ako je podatak osjetljiv, bolje ga je poslati kroz POST body.

Create i Update

Kod API-ja se podaci za kreiranje i izmjenu najčešće šalju kao JSON u tijelu zahtjeva. ASP.NET Core model binding mehanizam može automatski povezati JSON s C# objektom.

Primjer JSON objekta:

```

{
    "id": 1,
    "title": "Osnove ASP.NET Core MVC-a",
    "description": "Kviz s pitanjima o controllerima, viewovima i modelima.",
    "categoryID": 3,
}

```

```
"category": {  
    "id": 3,  
    "name": "Web programiranje"  
}
```

Odgovarajuća C# klasa:

```
public class Quiz  
{  
    public int ID { get; set; }  
    public string Title { get; set; }  
    public string Description { get; set; }  
    public int CategoryID { get; set; }  
    public Category Category { get; set; }  
}
```

Model binding čita propertyje iz JSON-a i popunjava C# objekt. Tipovi se automatski pretvaraju gdje je moguće. Kompleksni propertyji, poput `Category`, također se mogu povezati.

Kod API metoda često se koristi `[FromBody]` kako bi bilo jasno da model dolazi iz tijela zahtjeva:

```
public async Task<ActionResult<QuizDTO>> Put(int id, [FromBody]  
Quiz model)
```

Validacija API zahtjeva

API metoda mora provjeriti jesu li podaci koji dolaze od klijenta ispravni prije spremanja u bazu. Kod jednostavnih scenarija dovoljne su standardne validacijske anotacije i `ModelState`.

Primjer:

```
if (!ModelState.IsValid)  
{  
    return BadRequest(ModelState);  
}
```

Ako controller ima `[ApiController]`, ASP.NET Core može automatski vratiti `400 Bad Request` kada model nije validan. Ipak, važno je razumjeti da validacija postoji i da API ne smije spremati neispravne podatke samo zato što su došli u JSON-u.

Za složenija pravila može se koristiti `FluentValidation`. On omogućuje da se validacija izdvoji iz controllera i definira čitljivim pravilima, primjerice: obavezno polje, minimalna duljina, uvjetna validacija ili posebna poruka greške.

Kreiranje zapisa — POST

`POST` metoda služi za stvaranje novog zapisa. Poziva se kao:

```
POST /api/quiz
```

Primjer:

```
[HttpPost]
public ActionResult<QuizDTO> Post([FromBody] Quiz model)
{
    if (!ModelState.IsValid)
    {
        return BadRequest(ModelState);
    }

    this._dbContext.Quizzes.Add(model);
    this._dbContext.SaveChanges();

    return CreatedAtAction(
        nameof(Get),
        new { id = model.ID },
        ToDTO(model));
}
```

`CreatedAtAction` vraća HTTP status `201 Created` i informaciju gdje se novokreirani resurs može dohvatiti.

Izmjena zapisa — PUT

`PUT` metoda služi za spremanje promjena na postojećem zapisu. Poziva se kao:

PUT /api/quiz/1

Primjer:

```
[HttpPut("{id}")]
public ActionResult<QuizDTO> Put(int id, [FromBody] Quiz model)
{
    if (id != model.ID)
    {
        return BadRequest();
    }

    var quiz = this._dbContext.Quizzes.FirstOrDefault(c =>
c.ID == id);

    if (quiz == null)
    {
        return NotFound();
    }

    quiz.Title = model.Title;
    quiz.Description = model.Description;
    quiz.CategoryID = model.CategoryID;

    this._dbContext.SaveChanges();

    return Ok(ToDTO(quiz));
}
```



Kod izmjene je sigurnije dohvatiti postojeći zapis iz baze i ručno mapirati dopuštena polja nego direktno spremi objekt koji je došao u zahtjevu.

Brisanje zapisa

Brisanje se u API-ju najčešće implementira pomoću **DELETE** metode. Poziv sadrži ID zapisa koji treba obrisati.

Primjer poziva:

```
DELETE /api/quiz/1
```

Primjer metode:

```
[HttpDelete("{id}")]
public IActionResult Delete(int id)
{
    var quiz = this._dbContext.Quizzes.FirstOrDefault(c =>
c.ID == id);

    if (quiz == null)
    {
        return NotFound();
    }

    this._dbContext.Quizzes.Remove(quiz);
    this._dbContext.SaveChanges();

    return Ok();
}
```

Ako aplikacija koristi soft delete, umjesto **Remove** treba postaviti polje poput **DeletedAt** :

```
quiz.DeletedAt = DateTime.UtcNow;
this._dbContext.SaveChanges();
```

Dropzone

Upload datoteka čest je zahtjev u web aplikacijama. Korisnik treba moći odabrati jednu ili više datoteka, a aplikacija ih treba spremiti na server i povezati s odgovarajućim entitetom. Dropzone je JavaScript komponenta koja

omogućuje intuitivan upload datoteka. Datoteke šalje asinkrono na URL definiran u formi. Server je odgovoran za:

- prihvrat datoteke
- validaciju datoteke
- spremanje datoteke na disk ili storage
- spremanje metapodataka u bazu
- povezivanje datoteke s entitetom

Originalni Dropzone projekt nije aktivno održavan kao ranije, ali i dalje može raditi za potrebe vježbe. Moguće je koristiti i održavani fork ili sličnu biblioteku.



Za ovu vježbu bitan je koncept asinkronog uploada i povezivanja datoteke s kvizom. Sama biblioteka može biti Dropzone ili kompatibilna alternativa.

Gdje spremati datoteke

Za ovu vježbu dovoljno je spremiti datoteke na lokalni disk aplikacije. U stvarnim aplikacijama treba razmisliti o drugačijem storageu:

- relacijska baza najčešće nije dobro mjesto za spremanje većih datoteka
- dokumentna baza može biti prihvatljiva za vrlo male dokumente
- za ozbiljnije aplikacije bolje je koristiti Azure Blob Storage, Amazon S3, Firebase Storage ili sličan servis
- lokalni disk postaje problem kod horizontalnog skaliranja jer više instanci aplikacije nemaju nužno iste datoteke

Ako aplikacija radi u više instanci, jedna instanca može spremiti datoteku lokalno, a drugi zahtjev može završiti na drugoj instanci koja tu datoteku nema. Zato je za produkcijske sustave bolje koristiti zajednički storage servis nego rješavati sinkronizaciju datoteka između instanci.

Model Attachment

Jedan kviz može imati više datoteka. Zato se uvodi nova klasa `Attachment`.

Primjer:

```
Attachment.cs
```

```
public class Attachment
{
    public int ID { get; set; }

    public int QuizID { get; set; }
    public Quiz Quiz { get; set; }

    public string FileName { get; set; }
    public string FilePath { get; set; }
    public string ContentType { get; set; }
    public long FileSize { get; set; }

    public DateTime CreatedAt { get; set; }
}
```

U `Quiz` klasu dodati kolekciju:

```
public List<Attachment> Attachments { get; set; }
```

Nakon izmjene modela potrebno je napraviti migraciju i osvježiti bazu.

```
dotnet ef migrations add AddQuizAttachments
dotnet ef database update
```

Dropzone na Edit formi

Upload datoteka treba biti dostupan samo na Edit formi, jer kod Create forme kviz još nema ID. Bez ID-a nije jasno uz koji zapis treba vezati datoteku.

Primjer forme:

```
<div class="row">
  <div class="col-md-4">
    <form asp-action="Edit">
      <input type="hidden" asp-for="ID" />
      <partial name="_CreateOrUpdate" />
    </form>
  </div>
```

```

<div class="col-md-6">
  <label class="control-label">Dokumenti</label>
  <form id="attachmentDz"
        asp-controller="Quiz"
        asp-action="UploadAttachment"
        asp-route-quizId="@Model.ID"
        enctype="multipart/form-data"
        class="dropzone">

  </form>

  <div id="attachmentList"></div>
</div>
</div>

```

Uključivanje skripti

Skripte treba uključiti u `Scripts` sekciju pravog viewa. Partial view ne može definirati `@section Scripts`.

```

@section Scripts {
  <link rel="stylesheet" href="~/lib/dropzone/dropzone.cs
s" />
  <script src="~/lib/dropzone/dropzone.js"></script>

  <script type="text/javascript">
    Dropzone.options.attachmentDz = {
      success: function (file, response) {
        loadAttachments();
      }
    };

    $(function () {
      loadAttachments();
    });

    function loadAttachments() {
      $("#attachmentList").load("@Url.Action("GetAtta
chments", "Quiz", new { quizId = Model.ID })");
    }
  </script>
}

```



```
</script>  
}
```

Upload akcija

Akcija prima ID kviza i datoteku. Dropzone šalje datoteke kao multipart form data.

```
[HttpPost]  
public IActionResult UploadAttachment(int quizId, IFormFile  
file)  
{  
    var quiz = this._dbContext.Quizzes.FirstOrDefault(c =>  
c.ID == quizId);  
  
    if (quiz == null)  
    {  
        return NotFound();  
    }  
  
    if (file == null || file.Length == 0)  
    {  
        return BadRequest();  
    }  
  
    var uploadsPath = Path.Combine(  
        Directory.GetCurrentDirectory(),  
        "wwwroot",  
        "uploads",  
        "quizzes",  
        quizId.ToString());  
  
    Directory.CreateDirectory(uploadsPath);  
  
    var fileName = Guid.NewGuid().ToString() + Path.GetExte  
nsion(file.FileName);  
    var filePath = Path.Combine(uploadsPath, fileName);  
  
    using (var stream = new FileStream(filePath, FileMode.C
```

```

    create())
    {
        file.CopyTo(stream);
    }

    var attachment = new Attachment
    {
        QuizID = quizId,
        FileName = file.FileName,
        FilePath = "/uploads/quizzes/" + quizId + "/" + fil
eName,
        ContentType = file.ContentType,
        FileSize = file.Length,
        CreatedAt = DateTime.UtcNow
    };

    this._dbContext.Attachments.Add(attachment);
    this._dbContext.SaveChanges();

    return Json(new { success = true });
}

```



U stvarnoj aplikaciji treba dodatno validirati ekstenziju, MIME tip, veličinu datoteke i naziv datoteke. Nikad se ne smije nekontrolirano vjerovati uploadanom sadržaju.

Popis datoteka

Popis uploadanih datoteka treba dohvatiti AJAX pozivom nakon učitavanja stranice i nakon svakog uspješnog uploada.

Controller akcija:

```

public IActionResult GetAttachments(int quizId)
{
    var attachments = this._dbContext.Attachments
        .Where(a => a.QuizID == quizId)
        .OrderByDescending(a => a.CreatedAt)
}

```

```

        .ToList();

        return PartialView("_AttachmentList", attachments);
    }

```

Partial view:

`_AttachmentList.cshhtml`

```

<table class="table table-sm">
    <thead>
        <tr>
            <th>Datoteka</th>
            <th>Veličina</th>
            <th>Akcija</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var attachment in Model)
        {
            <tr>
                <td>
                    <a href="@attachment.FilePath" target
                    = "_blank">@attachment.FileName</a>
                </td>
                <td>@attachment.FileSize</td>
                <td>
                    <button type="button"
                        class="btn btn-danger btn-sm"
                        onclick="deleteAttachment(@atta
                    chment.ID)">
                        Obriši
                    </button>
                </td>
            </tr>
        }
    </tbody>
</table>

```

Brisanje datoteka

Brisanje se može napraviti AJAX pozivom.

Controller akcija:

```
[HttpPost]
public IActionResult DeleteAttachment(int id)
{
    var attachment = this._dbContext.Attachments.FirstOrDefault(a => a.ID == id);

    if (attachment == null)
    {
        return NotFound();
    }

    var physicalPath = Path.Combine(
        Directory.GetCurrentDirectory(),
        "wwwroot",
        attachment.FilePath.TrimStart('/'));

    if (System.IO.File.Exists(physicalPath))
    {
        System.IO.File.Delete(physicalPath);
    }

    this._dbContext.Attachments.Remove(attachment);
    this._dbContext.SaveChanges();

    return Json(new { success = true });
}
```

JavaScript:

```
<script type="text/javascript">
    function deleteAttachment(id) {
        $.ajax({
            url: "@Url.Action("DeleteAttachment", "Quiz")",
            method: "POST",
```

```
        data: { id: id },
        success: function () {
            loadAttachments();
        }
    });
}
```

</script>

Autentikacija i autorizacija

Autentikacija i autorizacija rješavaju dva različita problema.

Autentikacija odgovara na pitanje:

┃ Tko je korisnik?

Autorizacija odgovara na pitanje:

┃ Smije li taj korisnik napraviti ovu akciju?

Primjeri:

- korisnik se prijavljuje emailom i lozinkom — autentikacija
- samo admin smije brisati kvizove — autorizacija
- samo prijavljeni korisnici smiju uređivati podatke — autorizacija
- Google login potvrđuje identitet korisnika — autentikacija

Autentikacija

ASP.NET Core ima ugrađeni Identity sustav za lokalnu registraciju, prijavu, odjavu, reset lozinke, korisnike, role i vanjske login providere. Kod kreiranja nove MVC aplikacije moguće je odabrati `Individual Accounts`. Time Visual Studio generira osnovnu konfiguraciju za Identity. Za postojeću aplikaciju potrebno je ručno uskladiti projekt.

Za autentikaciju se ne preporučuje pisati vlastiti sustav od nule. ASP.NET Core Identity već rješava hashiranje lozinki, salt, logout, reset lozinke, korisnike, role, claimove i 2FA integracije. Budući da je riječ o široko korištenom frameworku, sigurnosni problemi se brže pronalaze i ispravljaju nego u vlastitom ručno pisanom rješenju.

Osnovni koraci:

1. U Model projekt dodati `AppUser` klasu koja nasljeđuje `IdentityUser`
2. `QuizManagerDbContext` treba naslijediti `IdentityDbContext<AppUser>`
3. Instalirati potrebne NuGet pakete
4. Pokrenuti migracije
5. Uskladiti `Program.cs`
6. Uočiti i po potrebi generirati `Areas/Identity` datoteke
7. Uskladiti `_Layout.cshtml` i `_LoginPartial.cshtml`

AppUser

Primjer:

`AppUser.cs`

```
using Microsoft.AspNetCore.Identity;

public class AppUser : IdentityUser
{
}
```

DbContext

Primjer:

`QuizManagerDbContext.cs`

```
public class QuizManagerDbContext : IdentityDbContext<AppUser>
{
    public QuizManagerDbContext(DbContextOptions<QuizManagerDbContext> options)
        : base(options)
    {
    }

    public DbSet<Quiz> Quizzes { get; set; }
```

```
public DbSet<Category> Categories { get; set; }  
}
```

Potrebni paketi ovise o strukturi projekata, ali tipično uključuju:

- `Microsoft.AspNetCore.Identity.EntityFrameworkCore`
- `Microsoft.Extensions.Identity.Stores`
- `Microsoft.AspNetCore.Identity.UI`

Program.cs

Konfiguracija se razlikuje ovisno o verziji projekta, ali osnovna ideja je registrirati Identity i uključiti autentikacijski middleware.

Primjer:

```
builder.Services  
    .AddDefaultIdentity<AppUser>(options =>  
    {  
        options.SignIn.RequireConfirmedAccount = false;  
    })  
    .AddRoles<IdentityRole>()  
    .AddEntityFrameworkStores<QuizManagerDbContext>();  
  
builder.Services.AddControllersWithViews();  
builder.Services.AddRazorPages();
```

U pipeline treba dodati:

```
app.UseAuthentication();  
app.UseAuthorization();  
  
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");  
  
app.MapRazorPages();
```



`app.UseAuthentication()` mora biti prije `app.UseAuthorization()`. Ako redoslijed nije ispravan, autorizacija neće raditi očekivano.

Login partial

Ako se koristi vlastita `AppUser` klasa, treba provjeriti gdje se u generiranom kodu koristi `IdentityUser`. Ta mjesta treba zamijeniti s `AppUser`.

Posebno provjeriti:

- `_LoginPartial.cshtml`
- `Register.cshtml.cs`
- `ExternalLogin.cshtml.cs`
- eventualne Identity stranice koje su scaffoldane



Najlakši način za provjeru konfiguracije je kreirati novu praznu ASP.NET Core MVC aplikaciju s odabranom opcijom `Individual Accounts`, a zatim usporediti razlike u `Program.cs`, layoutu i Identity datotekama.

OAuth

OAuth omogućuje prijavu preko vanjskih servisa poput Googlea, Facebooka, GitHuba ili Microsofta. U ovoj vježbi dovoljno je omogućiti jedan vanjski login provider, primjerice Google ili Facebook.

Za vanjsku prijavu potrebno je:

- omogućiti HTTPS
- kreirati aplikaciju na Google ili Facebook developers portalu
- dobiti `ClientId`
- dobiti `ClientSecret`
- konfigurirati provider u aplikaciji
- testirati registraciju i prijavu

Pojednostavljeni OAuth flow

OAuth flow omogućuje da aplikacija ne mora sama provjeravati korisnikovu lozinku kod vanjskog providera.

Pojednostavljeni tijek:

1. Korisnik klikne "Login with Google" ili "Login with Microsoft"
2. Aplikacija preusmjeri korisnika na login stranicu providera
3. Korisnik se prijavi kod providera, ne u našoj aplikaciji
4. Provider vraća korisnika natrag na callback URL aplikacije s `authorization code` vrijednošću
5. Aplikacija server-to-server pozivom provjerava taj code kod providera
6. Ako je code valjan, aplikacija kreira lokalnu prijavu, najčešće authentication cookie

`ClientId` identificira aplikaciju kod providera. `ClientSecret` je tajna vrijednost kojom aplikacija dokazuje da smije verificirati authorization code.

`Scope` određuje koje podatke aplikacija traži od korisnika, primjerice email ili ime. Treba tražiti samo ono što aplikaciji stvarno treba. Što je scope veći, korisnik će teže prihvatiti consent screen.

Postavljanje SSL-a

Vanjski login provideri zahtijevaju HTTPS. U development okruženju Visual Studio najčešće automatski konfigurira HTTPS, ali treba provjeriti

`launchSettings.json`.

`Properties/launchSettings.json`

```
{
  "profiles": {
    "https": {
      "applicationUrl": "https://localhost:7001;http://localhost:5001"
    }
  }
}
```

Google login

Primjer konfiguracije:

```
builder.Services
    .AddAuthentication()
    .AddGoogle(options =>
    {
        options.ClientId = builder.Configuration["Authentic
ation:Google:ClientId"];
        options.ClientSecret = builder.Configuration["Authe
ntication:Google:ClientSecret"];
    });
```

Tajne podatke ne treba hardkodirati u kod. U developmentu ih je bolje spremiti u user secrets.

```
dotnet user-secrets set "Authentication:Google:ClientId"
"..."
dotnet user-secrets set "Authentication:Google:ClientSecre
t" "..."
```



Prave secret vrijednosti ne smiju se hardkodirati u kod niti commitati u javni repository. U developmentu je praktično koristiti user secrets, a u produkciji environment varijable, secret manager, credential store ili vault. Rotacija procurjelog ključa može imati posljedice, zato je bolje spriječiti curenje nego ga kasnije popravljati.

Facebook login

Primjer konfiguracije:

```
builder.Services
    .AddAuthentication()
    .AddFacebook(options =>
    {
        options.AppId = builder.Configuration["Authenticati
on:Facebook:AppId"];
        options.AppSecret = builder.Configuration["Authenti
```

```
cation:Facebook:AppSecret"];  
});
```

Passkey

Passkey je moderni način prijave koji koristi private/public key princip. Privatni ključ ostaje na korisnikovom uređaju, a provider ima javni ključ kojim može provjeriti potpisani login zahtjev.

Korisnik se može prijaviti biometrijom, PIN-om ili drugim lokalnim mehanizmom uređaja, ali privatni ključ ne napušta uređaj. Zato je passkey sigurniji i praktičniji od klasične lozinke, posebno za aplikacije koje žele smanjiti ovisnost o passwordima.

Proširenje osnovne AppUser tablice

Osnovna `AspNetUsers` tablica često nije dovoljna. Aplikacija može trebati dodatne podatke o korisniku, primjerice OIB, JMBG, JMBAG ili interni identifikator.

U ovoj vježbi `AppUser` treba proširiti poljima:

- `OIB`
- `JMBG`

Primjer:

```
public class AppUser : IdentityUser  
{  
    [Required]  
    [StringLength(11, MinimumLength = 11)]  
    [RegularExpression("^[0-9]*$")]  
    public string OIB { get; set; }  
  
    [Required]  
    [StringLength(13, MinimumLength = 13)]  
    [RegularExpression("^[0-9]*$")]  
    public string JMBG { get; set; }  
}
```

Nakon izmjene klase potrebno je napraviti migraciju:

```
dotnet ef migrations add ExtendAppUserWith0ibAndJmbg
dotnet ef database update
```



Prije primjene migracije provjeriti što je generirano. Ako migracija generira neočekivane promjene nad drugim Identity poljima, te promjene treba razumjeti ili ukloniti prije `database update`.

Scaffold Identity stranica

Identity UI se može generirati kroz Visual Studio:

1. Desni klik na Web projekt → `Add` → `New Scaffolded Item` → odabrati `Identity`
2. Odabrati postojeći layout ili ostaviti prazno ako se koristi Razor `_ViewStart`
3. Odabrati stranice:
 - `Account/Register`
 - `Account/ExternalLogin`
4. Odabrati postojeći `QuizManagerDbContext`

Mogući problemi:

- scaffolder može generirati duplikat `QuizManagerDbContext` klase u `Identity/Data` folderu
 - taj duplikat treba obrisati
- Visual Studio može javiti grešku zbog verzije code generation paketa
 - provjeriti verziju paketa `Microsoft.VisualStudio.Web.CodeGeneration.Design`

Register forma

Nakon scaffoldanja treba modificirati:

- `Register.cshtml`
- `Register.cshtml.cs`

U input model dodati polja:

```
[Required]
[StringLength(11, MinimumLength = 11)]
```

```

[RegularExpression("[0-9]*$", ErrorMessage = "0IB smije sa
državati samo brojeve.")]
[Display(Name = "0IB")]
public string 0IB { get; set; }

[Required]
[StringLength(13, MinimumLength = 13)]
[RegularExpression("[0-9]*$", ErrorMessage = "JMBG smije s
adržavati samo brojeve.")]
[Display(Name = "JMBG")]
public string JMBG { get; set; }

```

Kod kreiranja korisnika postaviti vrijednosti:

```

var user = new AppUser
{
    UserName = Input.Email,
    Email = Input.Email,
    0IB = Input.0IB,
    JMBG = Input.JMBG
};

```

U `Register.cshtml` dodati polja:

```

<div class="form-floating mb-3">
    <input asp-for="Input.0IB" class="form-control" />
    <label asp-for="Input.0IB"></label>
    <span asp-validation-for="Input.0IB" class="text-dange
r"></span>
</div>

<div class="form-floating mb-3">
    <input asp-for="Input.JMBG" class="form-control" />
    <label asp-for="Input.JMBG"></label>
    <span asp-validation-for="Input.JMBG" class="text-dange
r"></span>
</div>

```

ExternalLogin forma

Kod prve prijave vanjskim servisom korisnik često mora potvrditi email i dovršiti registraciju. Ta forma je u:

```
Areas/Identity/Pages/Account/ExternalLogin.cshtml
```

I pripadajući page model:

```
Areas/Identity/Pages/Account/ExternalLogin.cshtml.cs
```

Treba napraviti iste promjene:

- dodati `OIB`
- dodati `JMBG`
- dodati validaciju
- prikazati polja u formi
- kod kreiranja `AppUser` objekta spremiti vrijednosti

Autorizacija korisnika

Nakon što znamo tko je korisnik, možemo ograničiti što smije raditi. U ASP.NET Core MVC-u za to se koristi atribut `[Authorize]`.

Primjer na razini akcije:

```
[Authorize]
public IActionResult Create()
{
    return View();
}
```

Primjer na razini controllera:

```
[Authorize]
public class QuizController : Controller
{
    public IActionResult Index()
    {
    }
```

```
{  
    return View();  
}
```

Ako je controller zaštićen, ali neka akcija treba biti javna, koristi se

`[AllowAnonymous]`:

```
[AllowAnonymous]  
public IActionResult Index()  
{  
    return View();  
}
```

Pravila za QuizController

Za ovu vježbu:

- svi mogu pregledavati listu kvizova i koristiti pretragu
- svi mogu pregledavati detalje kviza
- samo prijavljeni korisnici mogu kreirati, uređivati i brisati
- kasnije se dodatno uvode role za precizniju kontrolu

Primjer:

```
public class QuizController : BaseController  
{  
    [AllowAnonymous]  
    public IActionResult Index()  
    {  
        return View();  
    }  
  
    [AllowAnonymous]  
    public IActionResult Details(int id)  
    {  
        return View();  
    }  
}
```

```

[Authorize]
public IActionResult Create()
{
    return View();
}

[Authorize]
[HttpPost]
public IActionResult Create(Quiz model)
{
    // ...
}

[Authorize]
public IActionResult Edit(int id)
{
    return View();
}

[Authorize]
[HttpPost]
public IActionResult Edit(Quiz model)
{
    // ...
}

[Authorize]
public IActionResult Delete(int id)
{
    // ...
}
}

```

Informacije o trenutnom korisniku

Često je potrebno znati koji je korisnik napravio neku promjenu. Primjer:

- tko je kreirao kviz

- tko je zadnji izmijenio kviz
- kome pripada zapis
- koja pravila vrijede za tog korisnika

U controlleru se može dohvatiti ID trenutnog korisnika preko

`UserManager<AppUser>` :

```
private readonly UserManager<AppUser> _userManager;

public QuizController(
    QuizManagerDbContext dbContext,
    UserManager<AppUser> userManager)
{
    this._dbContext = dbContext;
    this._userManager = userManager;
}

public IActionResult Index()
{
    var userId = this._userManager.GetUserId(base.User);

    return View();
}
```

Bolje je takav kod izdvojiti u bazni controller.

BaseController

Primjer:

```
public abstract class BaseController : Controller
{
    protected readonly UserManager<AppUser> UserManager;

    protected BaseController(UserManager<AppUser> userManager)
    {
        this.UserManager = userManager;
    }
}
```

```

        protected string UserId
        {
            get
            {
                return this.UserManager.GetUserId(this.User);
            }
        }
    }
}

```

`QuizController` zatim nasljeđuje `BaseController`:

```

public class QuizController : BaseController
{
    private readonly QuizManagerDbContext _dbContext;

    public QuizController(
        QuizManagerDbContext dbContext,
        UserManager<AppUser> userManager)
        : base(userManager)
    {
        this._dbContext = dbContext;
    }
}

```

Autorizacija po ulogama

Autorizacija po ulogama omogućuje da različiti prijavljeni korisnici imaju različita prava. Primjer:

- `Admin` smije brisati
- `Manager` smije uređivati
- obični korisnik smije samo pregledavati detalje
- anonimni korisnik smije vidjeti samo listu

Da bi role radile, potrebno je:

- omogućiti role u Identity konfiguraciji

- kreirati role u bazi
- dodijeliti korisnike u role
- označiti akcije odgovarajućim `[Authorize(Roles = "...")]` atributima

Omogućavanje rola

U `Program.cs` treba uključiti role:

```
builder.Services
    .AddDefaultIdentity<AppUser>(options =>
    {
        options.SignIn.RequireConfirmedAccount = false;
    })
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<QuizManagerDbContext>();
```

Ako se koristi `AddIdentity`, konfiguracija može izgledati drugačije, ali ideja je ista: Identity mora znati koristiti `IdentityRole`.

Seed rola

Role se mogu dodati ručno u bazu, ali je bolje napraviti seed.

Primjer:

```
public static async Task SeedRoles(IServiceProvider serviceProvider)
{
    var roleManager = serviceProvider.GetRequiredService<RoleManager<IdentityRole>>();

    string[] roles = { "Admin", "Manager" };

    foreach (var role in roles)
    {
        if (!await roleManager.RoleExistsAsync(role))
        {
            await roleManager.CreateAsync(new IdentityRole(
                role));
        }
    }
}
```

```
}  
}
```

Dodjela role korisniku može se napraviti ručno u bazi za potrebe vježbe ili kroz privremeni seed.



Nakon dodavanja korisnika u rolu potrebno je napraviti logout i ponovno login. Role se nalaze u korisničkim claimovima i često se neće vidjeti dok se prijava ne osvježi.

Autorizacijske anotacije

Primjeri:

```
[Authorize(Roles = "Admin")]  
public IActionResult Delete(int id)  
{  
    // ...  
}
```

```
[Authorize(Roles = "Admin,Manager")]  
public IActionResult Edit(int id)  
{  
    // ...  
}
```

```
[Authorize]  
public IActionResult Details(int id)  
{  
    // ...  
}
```

```
[AllowAnonymous]  
public IActionResult Index()  
{
```

```
// ...  
}
```

Pravila za vježbu

Za `QuizController` treba vrijediti:

Akcija	Pristup
<code>Index</code> i pretraga	Svi korisnici, uključujući anonimne
<code>Details</code>	Bilo koji prijavljeni korisnik
<code>Create</code>	<code>Admin</code> ili <code>Manager</code>
<code>Edit</code>	<code>Admin</code> ili <code>Manager</code>
<code>Delete</code>	Samo <code>Admin</code>

Primjer:

```
public class QuizController : BaseController  
{  
    [AllowAnonymous]  
    public IActionResult Index()  
    {  
        return View();  
    }  
  
    [Authorize]  
    public IActionResult Details(int id)  
    {  
        return View();  
    }  
  
    [Authorize(Roles = "Admin,Manager")]  
    public IActionResult Create()  
    {  
        return View();  
    }  
  
    [Authorize(Roles = "Admin,Manager")]  
    [HttpPost]
```

```

public IActionResult Create(Quiz model)
{
    // ...
}

[Authorize(Roles = "Admin,Manager")]
public IActionResult Edit(int id)
{
    return View();
}

[Authorize(Roles = "Admin,Manager")]
[HttpPost]
public IActionResult Edit(int id, Quiz model)
{
    // ...
}

[Authorize(Roles = "Admin")]
public IActionResult Delete(int id)
{
    return View();
}

[Authorize(Roles = "Admin")]
[HttpPost]
public IActionResult DeleteConfirmed(int id)
{
    // ...
}
}

```

Role vs permission

Role su dobre kada aplikacija ima mali broj jasnih tipova korisnika, primjerice

`Admin`, `Manager`, `Student` ili `Professor`.

Ako sustav počne imati mnogo sitnih pravila, role mogu eksplodirati u previše kombinacija. Tada se često uvode permissioni ili privilegei, primjerice:

- `tasks.edit`
- `quiz.delete`
- `users.manage`

Role odgovaraju na pitanje kojoj skupini korisnik pripada. Permissionsi odgovaraju na pitanje smije li korisnik napraviti konkretnu akciju.

Za ovu vježbu role su dovoljne. U većim sustavima permissionsi daju finiju kontrolu, ali zahtijevaju dodatnu infrastrukturu, primjerice custom authorization attribute ili policy.

Integracijski testovi API endpointa

Integracijski testovi ne služe tome da se napravi velika količina umjetnih testova koji samo prolaze. Cilj je dokazati da aplikacija radi kroz stvarni HTTP sloj: ruta, model binding, validacija, autorizacija, baza i JSON odgovor moraju raditi zajedno.

Cilj testova nije samo dokazati da metoda trenutno radi. Glavna vrijednost testova je regresijska zaštita: kada se kasnije promijeni kod, testovi brzo pokažu je li nešto neočekivano puklo.

Za ASP.NET Core se najčešće koristi `WebApplicationFactory`, ali sama factory klasa nije najvažniji dio. Najvažnije je kontrolirati testno okruženje: bazu, konfiguraciju, seed podataka i vanjske ovisnosti.

Primjer paketa:

- `Microsoft.AspNetCore.Mvc.Testing`
- `Microsoft.EntityFrameworkCore.InMemory`
- `xunit`
- `FluentAssertions` po želji

Prvo dokazati jedan end-to-end scenarij

Prije nego se AI-u zada da generira cijeli paket testova, treba **polu-ručno** složiti jedan kvalitetan integracijski test end-to-end.

Cilj nije odmah generirati 50 AI-generiranih testova, nego prvo dokazati obrazac:

1. odabrati jedan vertikalni scenarij, primjerice `GET /api/quiz/{id}` ili `POST /api/quiz`

2. ručno ili uz pažljivo vođenje AI-a složiti `WebApplicationFactory`, testnu konfiguraciju i testnu bazu
3. seedati minimalne podatke potrebne za taj scenarij
4. pozvati stvarni endpoint preko `HttpClient`
5. provjeriti HTTP status, JSON odgovor i stanje baze gdje je bitno
6. tek kada taj test radi i kod izgleda razumljivo, dati AI-u da isti obrazac replicira na ostale CRUD scenarije

Takav pristup smanjuje AI slop: prvi test definira arhitekturu, stil, helper metode i razinu provjere, a AI zatim širi već provjereni obrazac umjesto da izmišlja cijelu testnu strategiju odjednom.

Testna baza: EF InMemory

Kod aplikacija koje koriste Entity Framework u testovima je najjednostavnije koristiti InMemory provider. Time se izbjegava ovisnost o lokalnom SQL Serveru, connection stringovima i stanju razvojne baze.

Tipična ideja:

- svaki test dobije svoju InMemory bazu
- baza se napuni minimalnim podacima potrebnima za taj test
- test ne ovisi o redoslijedu pokretanja drugih testova
- test ne koristi stvarnu development bazu

Primjer registracije testne baze:

```
builder.ConfigureServices(services =>
{
    var descriptor = services.SingleOrDefault(
        d => d.ServiceType == typeof(DbContextOptions<QuizM
anagerDbContext>));

    if (descriptor != null)
    {
        services.Remove(descriptor);
    }

    services.AddDbContext<QuizManagerDbContext>(options =>
```



```
{
    options.UseInMemoryDatabase("QuizManagerTests");
});
});
```



InMemory baza nije ista kao SQL baza. Ne provjerava sve relacijske constraintove i može se ponašati drugačije kod složenijih queryja. Za ovu vježbu je dobra jer omogućuje fokus na API ponašanje, ali razliku treba razumjeti.

Override konfiguracije u testovima

Aplikacija često čita konfiguraciju iz `appsettings.json`, user secrets, environment varijabli ili connection stringova. Testovi ne smiju ovisiti o stvarnoj konfiguraciji.

U `WebApplicationFactory` se konfiguracija može nadjačati posebno za testove:

```
builder.ConfigureAppConfiguration((context, config) =>
{
    var testSettings = new Dictionary<string, string>
    {
        ["ConnectionStrings:DefaultConnection"] = "TestConn
ection",
        ["Authentication:Google:ClientId"] = "test-client-i
d",
        ["Authentication:Google:ClientSecret"] = "test-clie
nt-secret"
    };

    config.AddInMemoryCollection(testSettings);
});
```

Ovo je korisno kada aplikacija očekuje da neka vrijednost postoji, ali u testu ne želimo koristiti stvarnu vrijednost.

Vanjske integracije treba mockati

Ako aplikacija tijekom API poziva šalje email, poziva payment provider, Google, Facebook, storage servis ili bilo koju drugu 3rd party integraciju, test ne bi

trebao stvarno zvati taj servis.

Rješenje je takvu funkcionalnost sakriti iza interfacea:

```
public interface IEmailSender
{
    Task SendAsync(string to, string subject, string body);
}
```

U aplikaciji se registrira stvarna implementacija, a u testu mock ili fake implementacija:

```
builder.ConfigureServices(services =>
{
    services.AddSingleton<IEmailSender, FakeEmailSender>();
});
```



Mockati treba granice sustava: vanjske servise, email, file storage, payment gatewaye i slične ovisnosti. Ne treba mockati sve interne klase samo zato što se može.

Interface ima smisla kada postoji više implementacija ili kada u testu treba zamijeniti stvarnu vanjsku integraciju fake/mock implementacijom. Ako klasa ima samo jednu implementaciju i ne postoji potreba za zamjenom, interface često samo dodaje nepotrebnu složenost.

Ne testirati mockove umjesto aplikacije

Česta greška kod AI-generiranih testova je da se napravi previše mockova. Tada test zapravo provjerava da mock vraća ono što smo mu ručno rekli da vrati. Loš obrazac:

- ✗ mockati repozitorij
- ✗ mockati service
- ✗ mockati mapper
- ✗ mockati validaciju
- ✗ zatim provjeriti da controller vraća rezultat iz mocka

Takav test ima malu vrijednost jer ne provjerava stvarno ponašanje aplikacije. Bolji obrazac:

- ✓ pokrenuti aplikaciju kroz `WebApplicationFactory`
- ✓ koristiti pravi `DbContext` s InMemory bazom
- ✓ napuniti podatke pomoćnim metodama
- ✓ pozvati API preko `HttpClient`
- ✓ provjeriti HTTP status i podatke u odgovoru

Primjer pomoćne metode:

```
private async Task<Quiz> CreateQuizAsync(QuizManagerDbContext dbContext)
{
    var quiz = new Quiz
    {
        Title = "Test",
        Description = "Test quiz"
    };

    dbContext.Quizzes.Add(quiz);
    await dbContext.SaveChangesAsync();

    return quiz;
}
```

Test tada jasno pokazuje što priprema, što poziva i što očekuje.

Što minimalno testirati

Za svaki API controller treba pokriti osnovne scenarije:

- ☐ `GET all` vraća uspješan status i kolekciju
- ☐ `GET by id` vraća zapis ako postoji
- ☐ `GET by id` vraća `404` ako zapis ne postoji
- ☐ `POST` kreira zapis i vraća `201 Created`
- ☐ `POST` vraća grešku za validacijski neispravan model
- ☐ `PUT` mijenja postojeći zapis
- ☐ `PUT` vraća grešku za nepostojeći zapis
- ☐ `DELETE` briše postojeći zapis

- ☐ **DELETE** vraća grešku za nepostojeći zapis
- ☐ zaštićeni endpointi vraćaju odgovarajući status ako korisnik nije autoriziran

Dobar integracijski test

Dobar integracijski test ima jasnu strukturu:

1. **Arrange** — pripremi testnu bazu, konfiguraciju i potrebne podatke
2. **Act** — pozovi stvarni API endpoint preko `HttpClient`
3. **Assert** — provjeri status kod, JSON odgovor i stanje baze ako je potrebno

Primjer:

```
[Fact]
public async Task GetById_ShouldReturnQuiz_WhenQuizExists()
{
    using var scope = this._factory.Services.CreateScope();
    var dbContext = scope.ServiceProvider.GetRequiredService<QuizManagerDbContext>();
    var quiz = await CreateQuizAsync(dbContext);

    var response = await this._client.GetAsync($"/api/quiz/{quiz.ID}");

    response.StatusCode.Should().Be(HttpStatusCode.OK);

    var dto = await response.Content.ReadFromJsonAsync<QuizDTO>();
    dto.ID.Should().Be(quiz.ID);
    dto.Title.Should().Be(quiz.Title);
}
```